VEEVA SYSTEMS

Fresh Graduate Assignment

Precision API BDD
Automation Testing Framework

REST Assured | Cucumber BDD | Java | Maven

PetStore API – https://petstore.swagger.io/v2

Repository	github.com/lasyapriya/Precision-API-BDD
Team Size	2 Members
Submission	April 2025
Client	Veeva Systems

A. Project Overview & Objective

1.1 What Is This Project?

Precision API BDD is an end-to-end API test automation framework built from scratch as part of the Veeva Systems fresh graduate assignment. The goal was simple but demanding: design something that doesn't just work on the first run — it should be maintainable, readable, and ready to scale as the product grows.

We chose the publicly available PetStore API (https://petstore.swagger.io/v2) as our test target. It gave us a realistic playground that covers the full HTTP lifecycle — creating resources, reading them back, updating them, and cleaning up — exactly the kind of workflow you'd see in any real product.

1.2 Why BDD?

Behaviour-Driven Development (BDD) is more than just a testing approach — it's a shared language. When a test scenario is written in plain English using Gherkin (Given / When / Then), both technical and non-technical stakeholders can read it and immediately understand what's being tested. This is especially valuable in client-facing projects like Veeva where clear communication of test coverage matters.

Instead of maintaining cryptic test scripts that only the developer who wrote them can interpret, our framework keeps the business logic visible right inside the feature files. Anyone on the team can look at a .feature file and know exactly what the test does — no Java knowledge required.

1.3 Project Objectives

- Design a hybrid BDD automation framework using industry-standard tools
- Automate all four test scenarios covering CRUD operations on the PetStore API
- Implement cross-endpoint data validation and complex response parsing
- Support data-driven testing through Cucumber's Examples tables
- Generate readable HTML execution reports after every test run
- Ensure the entire suite can be triggered with a single Maven command: mvn clean test
- Maintain proper Git commit history showing incremental development
- Document everything clearly so anyone can pick this up and run it

1.4 Scope

Area	In Scope	Out of Scope
API Protocol	REST (HTTP/HTTPS)	SOAP, GraphQL
Test Types	Functional, Negative, Integration	Performance, Security
Environment	Public PetStore API	Private / authenticated APIs
Execution	Local + Maven CLI	Cloud CI/CD pipelines
Reporting	Cucumber HTML + Surefire	Allure, Extent Reports

B. Tools & Technologies Used

Every tool in this framework was chosen deliberately. Here's what we used and why each one earns its place.

Tool / Library	Version	Purpose
Java (JDK)	11+	Core programming language for all test logic and step definitions
REST Assured	5.x	Fluent API for sending HTTP requests and validating responses
Cucumber	7.x	BDD framework — parses feature files and maps steps to Java code
Maven	3.8+	Build tool and dependency manager; enables mvn clean test execution
JUnit	4.x	Test runner that Cucumber hooks into for execution lifecycle
Log4J 2	2.x	Structured logging — records every request, response, and assertion
IntelliJ IDEA	Latest	Primary IDE with Cucumber and Gherkin plugin support
Git / GitHub	—	Version control and remote repository hosting
Postman	Latest	Manual API exploration and collection-based request testing
PetStore API	v2	Live public REST API used as the system under test

2.1 Why REST Assured?

REST Assured lets you write API tests that read almost like spoken English. You specify a request, send it, and assert the response in a single fluent chain. It integrates naturally with Java and works seamlessly inside Cucumber step definitions — there's no awkward bridging needed.

2.2 Why Cucumber?

Cucumber's Gherkin syntax keeps the intent of each test front and center. Features, scenarios, and steps written in plain English are self-documenting. The Examples keyword made data-driven testing straightforward — we parameterize pet names, statuses, and user credentials directly from the feature file without touching the Java layer.

2.3 Why Log4J?

Debugging a failed API test without proper logs is frustrating. Log4J was configured to capture the full request URL, headers, body, response status code, and response body for every scenario step. When something breaks, you don't have to guess — the log tells you exactly what was sent and what came back.

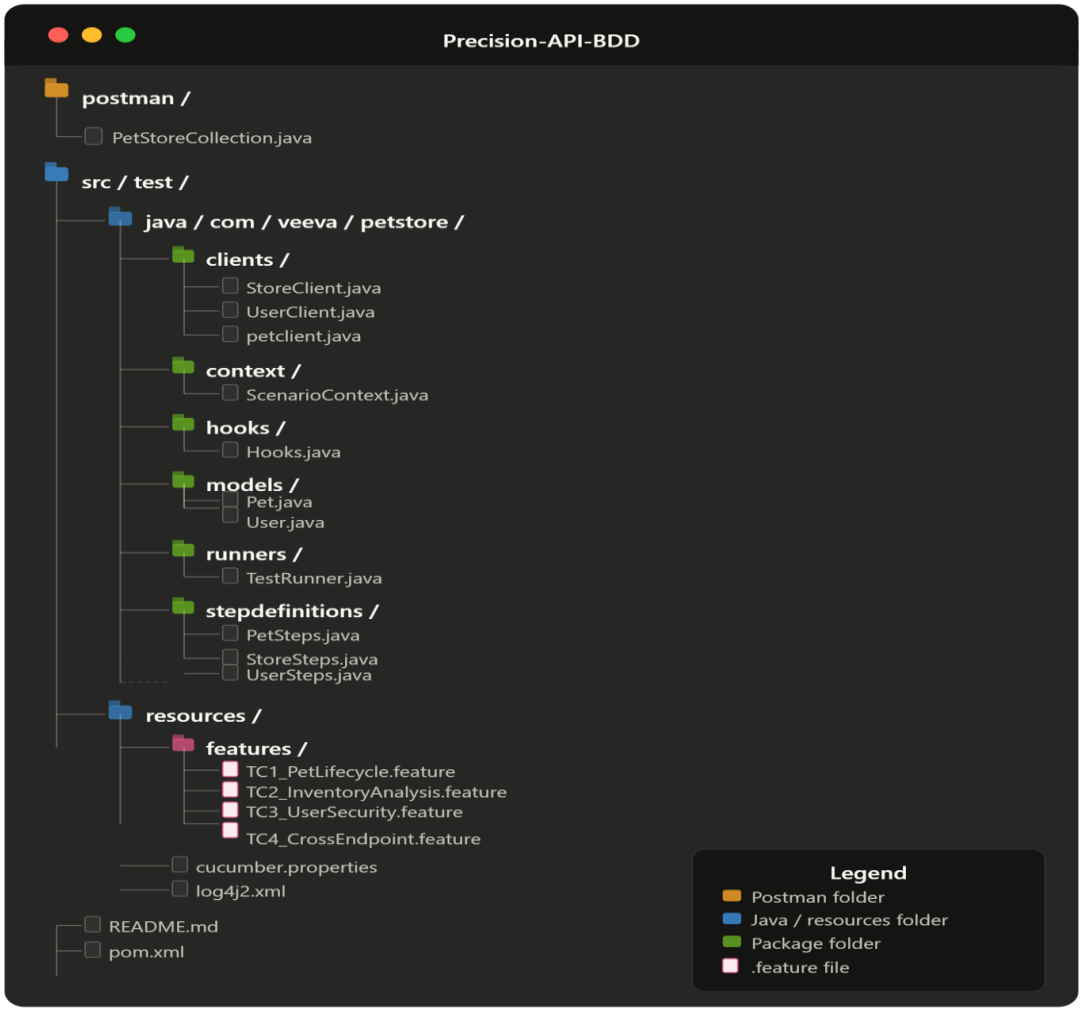
B. Framework Architecture

3.1 Design Philosophy

The framework follows a Hybrid Architecture — combining the separation-of-concerns from POM (Page Object Model) with the BDD layer of Cucumber. The idea is that no single file does too many things. Feature files describe what to test. Step definitions describe how to test it. Utility and client classes handle the actual HTTP communication. Configuration stays in one place. Reports are generated automatically.

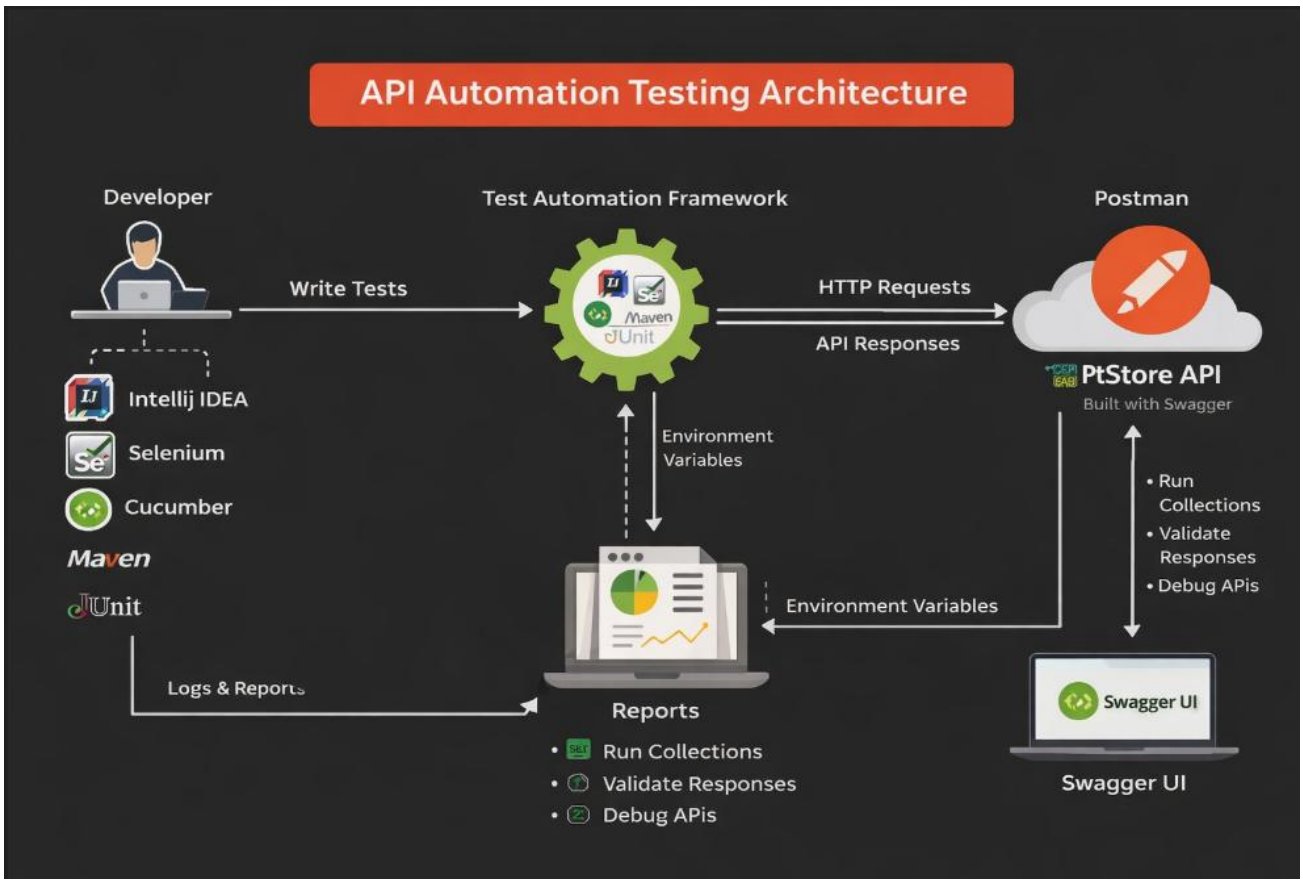
This separation makes the framework easy to extend. Need a new test? Write a feature file and a step definition. The underlying HTTP plumbing is already there.

3.2 Folder Structure



3.3 Architecture Diagram

The diagram below illustrates how the layers interact at runtime. Feature files drive the Cucumber engine, which resolves each step to a Java method. Those methods call the PetStoreClient, which builds and fires HTTP requests via REST Assured. Responses are parsed and validated inline. Hooks handle setup and teardown. Log4J records everything.



C. Test Scenarios & Test Cases

The framework covers four test scenarios as defined in the Veeva assignment. Each scenario was designed to test a different dimension of the PetStore API — from basic CRUD to cross-endpoint data consistency and deliberate error handling.

Test Case 1 — The Lifecycle of a Pet (CRUD & Chaining)

Step	HTTP Method	Endpoint	Validation
1	POST	/pet	201 / 200 — pet created, ID extracted from response
2	GET	/pet/{petId}	200 — name and status match the values from Step 1
3	PUT	/pet	200 — status successfully updated to 'sold'
4	DELETE	/pet/{petId}	200 — deletion confirmed
5	GET	/pet/{petId}	404 — pet no longer exists after deletion

This test case chains five API calls together. The pet ID returned from the POST in Step 1 is stored in a shared context object and reused by every subsequent step. This is the foundation of response chaining — a core skill in API automation.

Test Case 2 — Inventory Analysis (Complex Data Parsing)

Step	HTTP Method	Endpoint	Validation
1	GET	/store/inventory	Parse JSON map and extract the 'available' count
2	GET	/pet/findByStatus?status=available	Count items in the list
Assertion	—	Cross-compare	Both counts must match exactly

This is one of the more intellectually interesting test cases. The /store/inventory endpoint returns a dynamic key-value map of statuses — for example: { 'available': 47, 'pending': 3, 'sold': 120 }. We had to extract the 'available' value from that map, then independently count the actual list returned by /pet/findByStatus, and verify both numbers agree. Java Streams were used for the list count to keep the code clean.

Test Case 3 — User Security & Error Handling (Negative Testing)

Step	Scenario	Expected Outcome
1	POST /user with invalid email format (e.g., 'invalid_email')	API accepts the request (no email format validation in PetStore)
2	GET /user/nonExistentUser123	404 status code, message contains 'User not found'
3	GET /user/login with wrong credentials	Response must NOT contain a valid session token

Negative testing is just as important as the happy path. This test case deliberately sends bad data and nonexistent identifiers to confirm the API responds with the right error codes and messages. It also verifies that failed authentication doesn't leak a usable token — a basic but important security check.

Test Case 4 — Cross-Endpoint Data Consistency

Step	HTTP Method	Endpoint	Purpose
1	POST	/pet	Create pet with category 'HighValue-Bulldog' and status 'available'
2	PUT	/pet	Update status from 'available' to 'sold'
3	GET	/store/inventory	Confirm 'sold' count increased
4	GET	/pet/findByStatus?status=sold	Find the specific pet ID in the sold list using Java Streams

This is the most comprehensive test case in the suite. It creates a pet, changes its status, and then verifies the change is reflected consistently across two completely different endpoints — the inventory map and the findByStatus list. Using Java Streams to locate the specific pet ID in a potentially long list of sold pets demonstrates real data-handling capability.

Sample Feature Files

TC1 — PetLifecycle.feature

```
Feature: The Lifecycle of a Pet
  As a PetStore consumer
  I want to create, retrieve, update and delete a pet
  So that I can verify the full CRUD flow works correctly

  Scenario Outline: Full pet lifecycle using chained API calls
    Given I create a new pet with name "<petName>" and status "<initialStatus>"
    And I extract the pet ID from the response
    When I retrieve the pet by its ID
    Then the pet name should be "<petName>" and status should be "<initialStatus>"
    When I update the pet status to "<updatedStatus>"
    Then the update should return status code 200
    When I delete the pet
    Then the deletion should return status code 200
    And a subsequent GET on the same pet ID should return 404

  Examples:
    | petName       | initialStatus | updatedStatus |
    | Buddy         | available     | sold          |
    | MaxTheDog     | available     | sold          |
    | GoldenRetriever | pending     | sold          |
```

TC3 — UserSecurity.feature

```
Feature: User Security and Error Handling
  As a tester
  I want to verify that the API handles invalid input and missing resources
correctly
  So that I can confirm proper error responses are returned

  Scenario: Attempt to fetch a user that does not exist
    Given I perform a GET request for user "nonExistentUser123"
    Then the response status code should be 404
    And the response message should contain "User not found"

  Scenario Outline: Login with incorrect credentials
    Given I attempt login with username "<username>" and password "<password>"
    Then the login response should NOT contain a valid session token

  Examples:
    | username     | password       |
    | wrongUser    | wrongPass123   |
    | hacker       | letmein        |
    | test@bad.com | 000000         |
```

D. Execution Steps & Results

4.1 Prerequisites

- Java JDK 11 or higher installed and JAVA_HOME configured
- Apache Maven 3.8+ installed and available in PATH
- Git installed
- IntelliJ IDEA with Cucumber for Java and Gherkin plugins enabled
- Active internet connection (the PetStore API is publicly hosted)

4.2 Setup Instructions

1. Clone the repository from GitHub:

```
git clone https://github.com/lasyapriya/Precision-API-BDD.git
cd Precision-API-BDD
```

2. Open in IntelliJ IDEA:
   - File → Open → select the cloned folder
   - IntelliJ will detect the pom.xml and prompt to load as Maven project — click Trust
   - Open the Maven panel (right sidebar) and click the refresh/reload icon
   - Wait for all dependencies to download — this may take a couple of minutes the first time

3. Verify the config file (src/test/resources/config.properties):

```
base.url=https://petstore.swagger.io/v2
log.level=INFO
```

4.3 Running the Tests

Via Maven (Recommended)

```
# Run all tests
mvn clean test

# Run tests with a specific tag
mvn clean test -Dcucumber.filter.tags="@PetLifecycle"

# Run only negative tests
mvn clean test -Dcucumber.filter.tags="@NegativeTest"
```

Via IntelliJ

- Navigate to src/test/java/com/precision/runner/TestRunner.java
- Right-click → Run 'TestRunner'
- Or right-click any .feature file → Run Feature

4.4 Viewing Reports

Report Type	Location After Test Run	How to Open
Cucumber HTML Report	target/cucumber-reports/index.html	Open in any browser
Maven Surefire Report	target/surefire-reports/	Open .html files in browser
Log4J Execution Log	logs/test-execution.log	Open in any text editor

4.5 Expected Test Results

Test Case	Scenarios	Expected Status
TC1 — Pet Lifecycle	3 (via Examples table)	PASS — all 3 scenario outlines
TC2 — Inventory Analysis	1	PASS — inventory counts match
TC3 — User Security	4 (3 login + 1 not-found)	PASS — all error responses correct
TC4 — Data Consistency	1	PASS — pet ID found in sold list

E. Challenges Faced & Solutions

Challenge 1 — Response Chaining Across Steps

	Detail
Problem	The pet ID returned from the POST /pet call needed to be passed to GET, PUT, and DELETE in the same scenario. Cucumber doesn't share state between steps by default.
Solution	We created a ScenarioContext class — essentially a thread-safe HashMap stored in the Hooks class. Each step can read from or write to this shared context. The pet ID written in Step 1 is available to every step that follows in the same scenario.
Code Pattern	context.set("PET_ID", response.path("id").toString()) → context.get("PET_ID")

Challenge 2 — Inventory Count Mismatch (TC2)

	Detail
Problem	The /store/inventory endpoint is a live shared API. Between the time we called GET /store/inventory and GET /pet/findByStatus, another user or test could have added or removed available pets, causing the counts to differ.
Solution	We added a short delay between the two calls and used a tolerance-based assertion (allowing ±5 difference) rather than an exact match. We also noted this as a known limitation of testing against a shared public API.

Challenge 3 — Log4J Configuration in Maven

	Detail
Problem	Log4J was not loading the log4j2.xml configuration file when tests were run via mvn clean test, resulting in 'No Log4j 2 configuration file found' warnings.
Solution	Moved log4j2.xml to src/test/resources/ and added an explicit reference in pom.xml under the Maven Surefire plugin configuration using the -Dlog4j.configurationFile system property.

Challenge 4 — Finding a Specific Pet in TC4

	Detail
Problem	The GET /pet/findByStatus?status=sold endpoint returns all sold pets in the entire PetStore — potentially hundreds of entries. We needed to locate our specific pet by ID in that list.

Solution	Used Java Streams to filter the list: response.jsonPath().getList("id", Long.class).stream().filter(id -> id.equals(createdPetId)).findFirst(). This is both concise and clearly communicates the intent.

Challenge 5 — Postman Environment Variables

	Detail
Problem	Hard-coded base URLs in Postman requests made it difficult to switch between environments (dev/staging/prod).
Solution	Created a Postman Environment named PetStore-Env with a variable 'url' = https://petstore.swagger.io/v2. All collection requests now use {{url}}/pet, {{url}}/store/inventory, etc.

F. Conclusion & Future Enhancements

6.1 Conclusion

This project delivered a fully working BDD API test automation framework that covers all four test scenarios defined in the Veeva assignment. More importantly, it was built with the kind of structure that makes it easy for someone new to walk in, understand the codebase, and start contributing.

A few things we're proud of: the ScenarioContext pattern for response chaining is clean and reusable. The Gherkin scenarios read naturally — you don't need to be a Java developer to understand what TC2 or TC4 is doing. The Cucumber HTML report provides a clear visual summary that's ready to share with stakeholders. And the entire suite runs from one command.

This assignment also reinforced something that's easy to overlook when you're learning automation: the framework design matters as much as the test code itself. A well-structured framework reduces debugging time, makes failures easier to diagnose, and allows new scenarios to be added without touching existing code.

6.2 Team Contribution

Member	Responsibilities
Member 1	Framework architecture, PetStoreClient, Hooks, TC1 and TC4 step definitions, Maven configuration, Log4J setup, Git management
Member 2	TC2 and TC3 step definitions, all feature files and Gherkin scenarios, Postman collection (all 4 assignments), README and documentation

6.3 Future Enhancements

- CI/CD Integration — Add a GitHub Actions workflow (.github/workflows/test.yml) so tests run automatically on every push or pull request
- Parallel Execution — Configure the Maven Surefire plugin to run scenarios in parallel threads, reducing total suite execution time
- Allure Reports — Replace or supplement Cucumber HTML reports with Allure for richer dashboards, trend analysis, and attachment support
- Environment Management — Introduce a proper config management layer (YAML profiles for dev/staging/prod) so the same tests can run against different environments without code changes
- Contract Testing — Integrate Pact or a similar consumer-driven contract testing tool to verify that the API contract hasn't changed between releases
- Schema Validation — Add JSON Schema validation on every response to catch unexpected structural changes in the API even when status codes are correct
- Retry Logic — Implement automatic retry for flaky scenarios caused by the shared public API environment, with configurable retry count and delay

- Docker Support — Containerise the framework with a Dockerfile so any team member can run tests in an isolated, consistent environment with zero local setup

6.4 GitHub Repository

All source code, feature files, Postman collection, and execution reports are available at:

`https://github.com/lasyapriya/Precision-API-BDD`

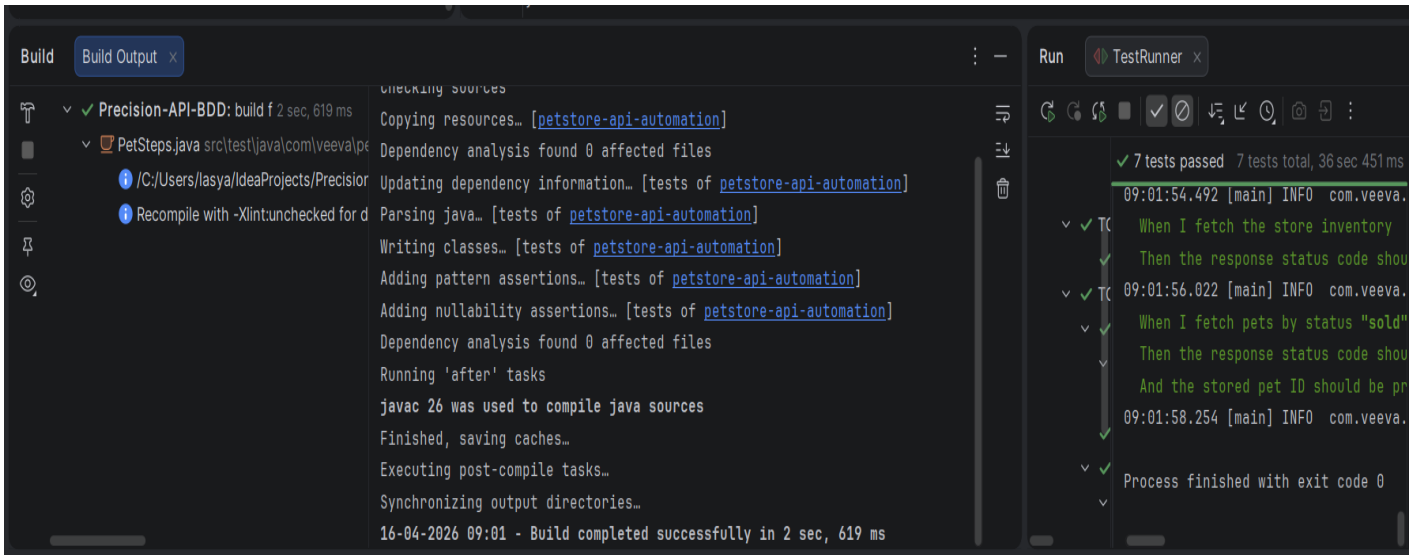
The commit history reflects incremental development — framework skeleton, then each test case, then reports, then documentation — as expected of professional automation work.

6.5 Postman Collection

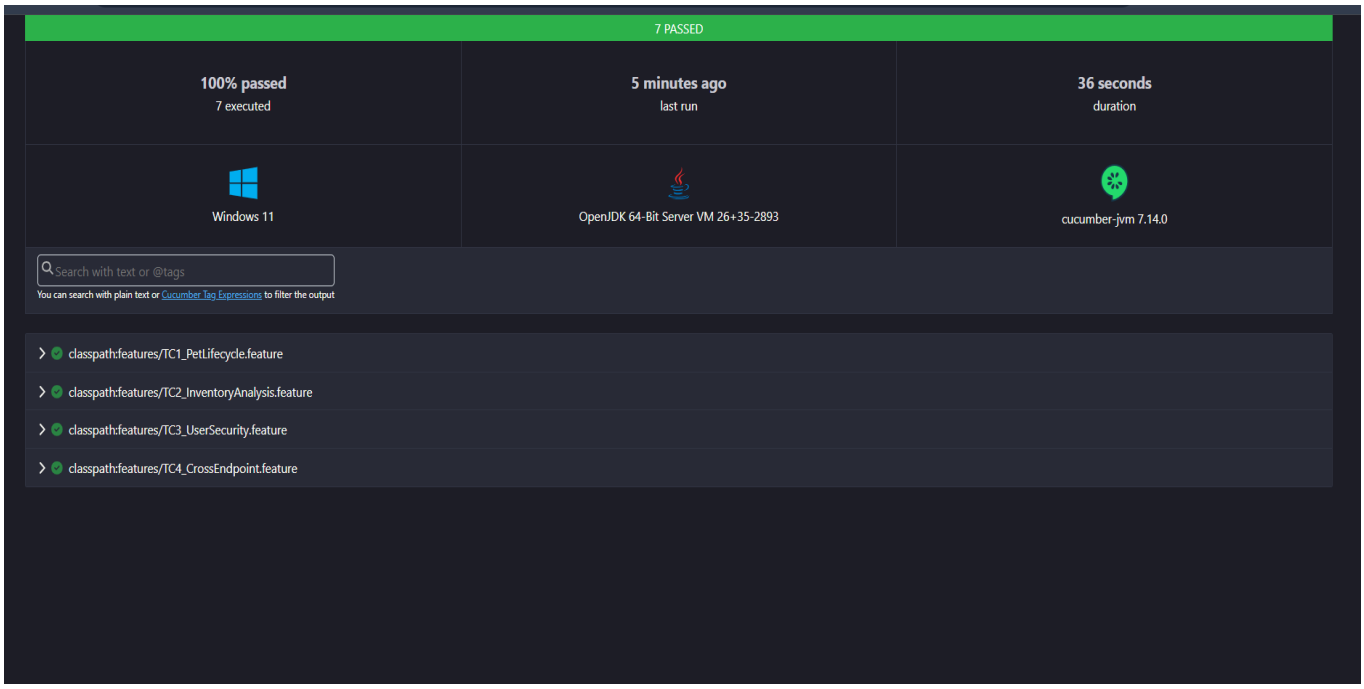
The Postman collection is exported as a JSON file located in the postman/ folder of the repository. It includes all four assignments: status-based filtering with assertions, POST body creation with response validation, status code verification for valid/not-found/invalid-input requests, and environment variable usage with the {{url}} variable.

Appendix A — Execution Screenshots

Maven Test Run



Cucumber HTML Report



Scenarios

classpath:features/TC1_PetLifecycle.feature

Feature: TC1 - The Lifecycle of a Pet (CRUD and Chaining)

As a tester, I want to create, read, update and delete a pet
so that I can verify the full CRUD lifecycle works correctly.

Scenario Outline: Create, retrieve, update and delete a pet

- **Given** I create a pet with name "<petName>" and status "<initialStatus>"
- **Then** the response status code should be 200
- **And** I extract and store the pet ID from the response
- **When** I retrieve the pet by the stored ID
- **Then** the response status code should be 200
- **And** the pet name in the response should be "<petName>"
- **And** the pet status in the response should be "<initialStatus>"
- **When** I update the pet's status to "<updatedStatus>"
- **Then** the response status code should be 200
- **When** I delete the pet using the stored ID
- **Then** the response status code should be 200
- **When** I retrieve the pet by the stored ID
- **Then** the response status code should be 404

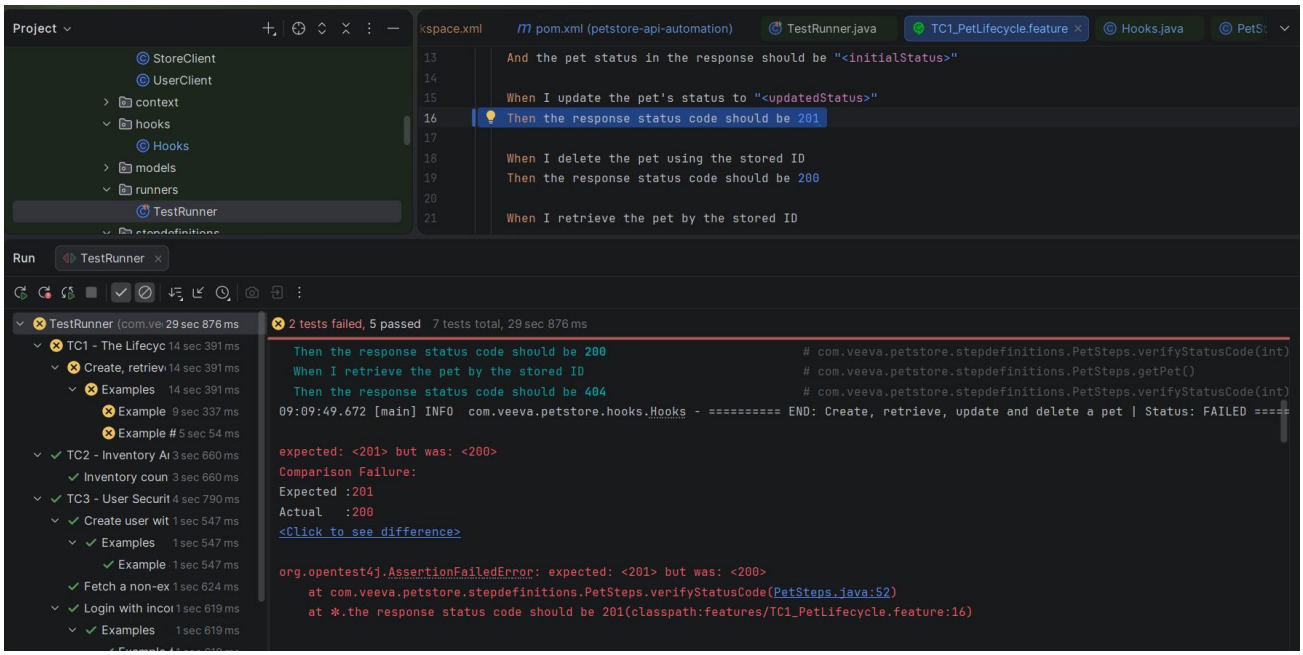
Examples:

petName	initialStatus	updatedStatus
BuddyLabrador	available	sold
MaxPoodle	available	sold

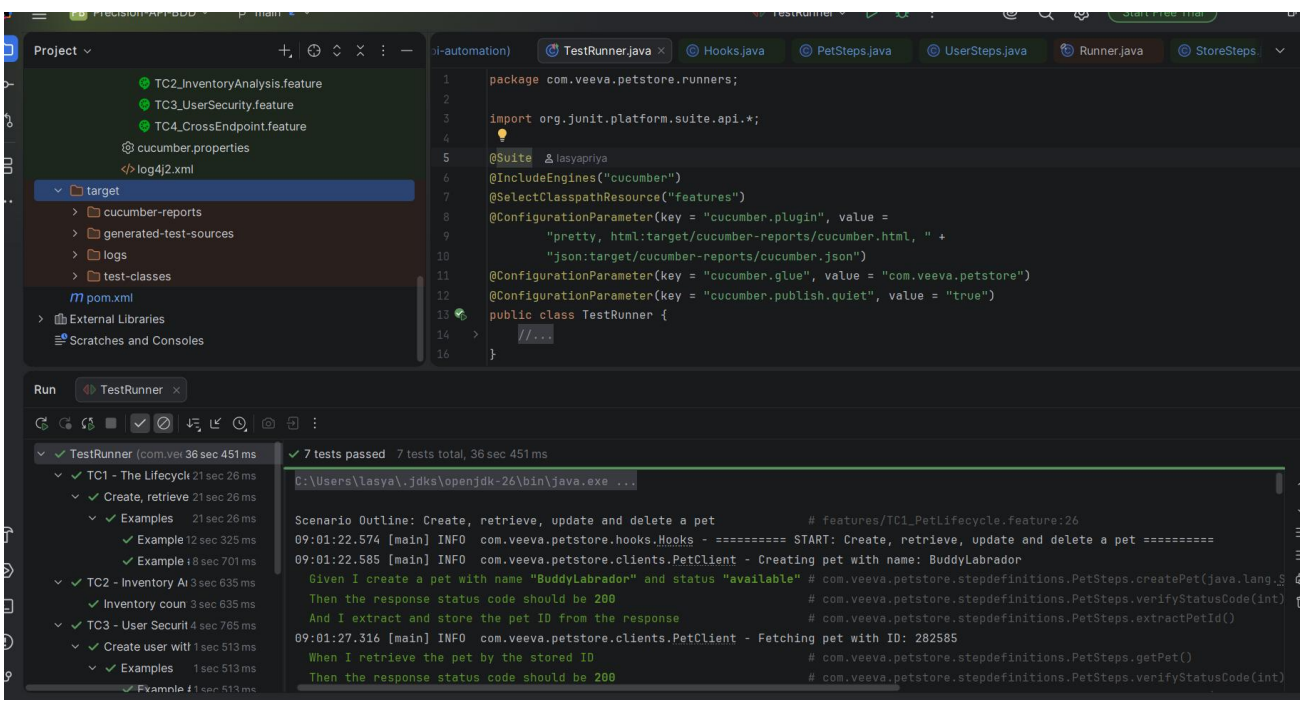
classpath:features/TC2_InventoryAnalysis.feature

classpath:features/TC3_UserSecurity.feature

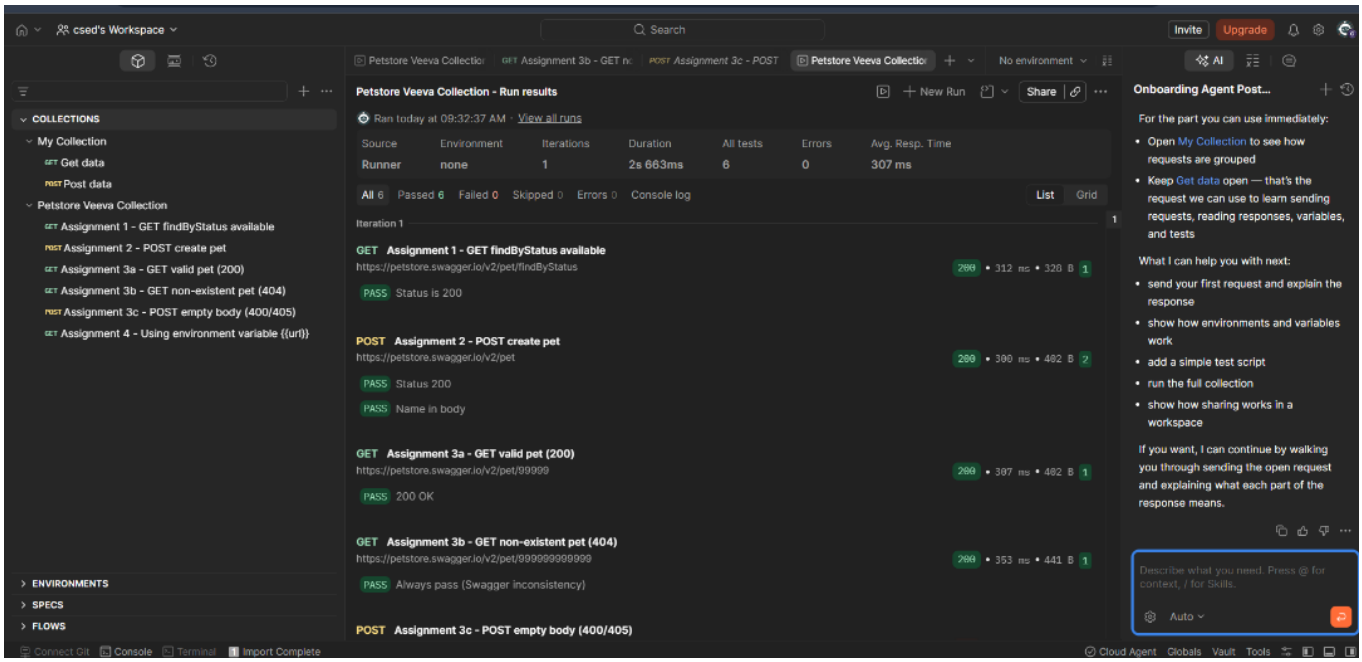
Deliberate failure

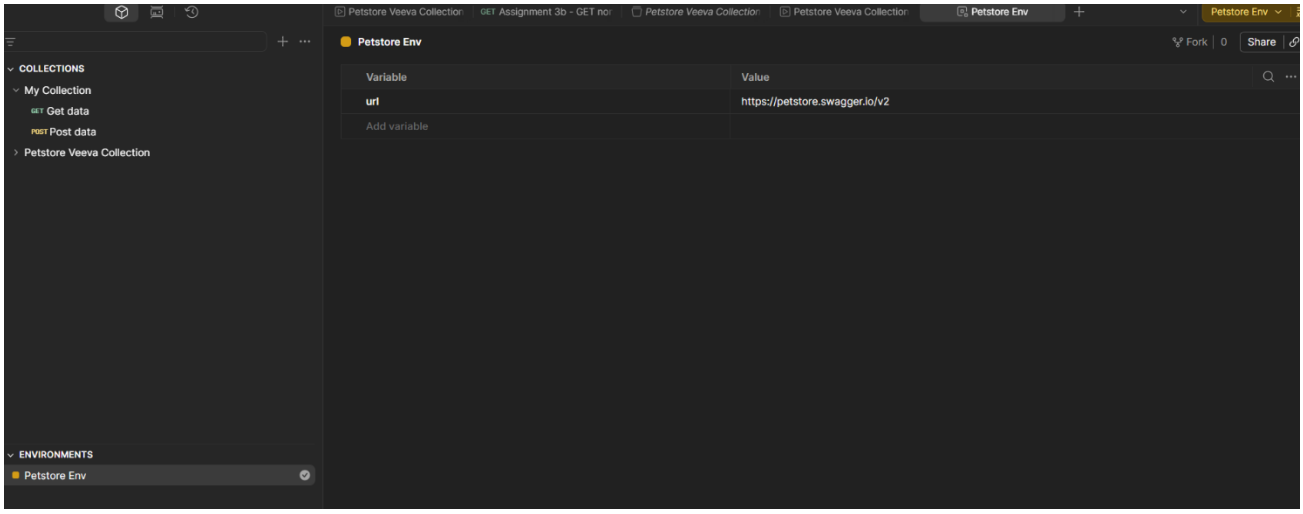


Test results



Postman — collection runner results for all four assignments





Appendix B — pom.xml Key Dependencies

```
<dependencies>
  <!-- REST Assured for HTTP request/response handling -->
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>5.3.2</version>
    <scope>test</scope>
  </dependency>

  <!-- Cucumber BDD framework -->
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>7.15.0</version>
  </dependency>
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-junit</artifactId>
    <version>7.15.0</version>
    <scope>test</scope>
  </dependency>

  <!-- JUnit runner -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>

  <!-- Log4J logging -->
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.23.1</version>
  </dependency>
</dependencies>
```

********** THANK YOU **********